

Compute Systems Engine: 代码 实践卷

从 reference 到可恢复分布式计算引擎

CPU Performance Study

本卷是《从 C++ 到计算系统：第二册》的配套代码实践卷，专门承接完整 C++20 实现、测试、benchmark、故障注入和实验报告。

前言

第二册原理卷负责回答“为什么”：为什么热循环会被分支、依赖和内存等待限制，为什么 false sharing 会让正确程序扩展失败，为什么条件变量必须围绕等待谓词，为什么异步 I/O 的 submit、completion 和 commit 不是同一件事，为什么分布式系统必须区分逻辑任务和物理 attempt。本卷负责回答“怎样落地”：怎样把这些原则写成一个可以构建、测试、测量、故障注入和复盘的 C++20 工程。

本卷的贯穿项目统一称为 **Compute Systems Engine**（计算系统引擎）。它从一个日志统计任务开始：输入是一批记录，输出是按 key 聚合的稳定结果。第一阶段只要求顺序 reference、输入合同和诊断；后续逐步加入 split、batch、冷热字段、benchmark、并行内核、线程池、有界队列、原子协议、任务图、可靠写、manifest、checkpoint、MessageBus、attempt、epoch、分片和故障注入。每一步都必须有测试和报告，不接受只写代码、不解释边界的实现。

本卷不是代码片段合集，而是一部工程教材。每章都遵循同一条路线：先说明本章要兑现原理卷中的哪个机制；再给出接口和目录组织；随后实现最小版本、测试失败输入、补充报告字段；最后说明这个版本还不能处理什么，下一章为什么必须继续扩展。读者应当把代码、测试和报告看成一个整体：代码实现机制，测试固定语义，报告保存证据。

核心思想

代码实践卷的目标不是写一个庞大的教学框架，而是让每一项复杂度都有购买理由：它修复了哪个失败，保留了哪个正确性边界，带来了什么运行成本，怎样用测试和报告证明。

常见缩写和术语先导

代码实践卷会把第二册的概念落到工程对象。第一次看到下面这些词时，先把它理解成“代码里必须有边界的对象”，而不是抽象口号；每个对象都要能在代码、测试或报告里找到证据。

reference 参考实现。它可以慢，但必须清楚、确定、可复查，用来定义正确结果。

oracle 判定器。它比较 **reference** 和待测实现，判断输出是否满足同一语义和误差合同。

contract 合同。这里不是法律文件，而是代码边界的明确约定：输入是什么，输出是什么，错误怎样表达，哪些行为不承诺。

RecordId 记录身份。它把一条输入记录定位到 **input name**、**generation**、**byte offset** 和 **line number**，方便诊断和重放。

diagnostic 诊断记录。它不是普通日志，而是给测试、报告和修复定位使用的结构化错误事实。

checksum 校验和。它把输出内容压成可比较摘要，用来检查结果是否稳定、文件是否被截断或误读。

CPU Central Processing Unit，中央处理器。本卷的实验默认先落在本地 CPU 上，再逐步讨论 SIMD、线程和 I/O。

UI User Interface，用户界面。第一章提到 UI 时，重点是错误和诊断如何被用户看懂，而不是先做图形界面。

C++20 C++ 标准版本名。本卷代码以 C++20 为默认边界，原因是它同时提供现代类型、**span**、同步原语和较好的工程表达能力。

CMake C++ 工程常用构建系统。本卷用它组织 **target**、测试、**benchmark** 和工具程序。

std:: C++ 标准库命名空间前缀。本卷第一章里的 **std::vector**、**std::string**、**std::map**、**std::uint64_t** 都来自标准库；读代码时先问它们表达“拥有一组元素”“保存字节串”“按键查值”还是“固定宽度整数”。

InputCase/ReferenceResult/ParseDiagnostic 本卷第一章的项目内示例名。**InputCase** 表示一条测试输入；**ReferenceResult** 表示参考实现的输出；**ParseDiagnostic** 表示解析失败时的结构化诊断。它们不是行业缩写，而是代码合同的名字。

SIMD Single Instruction, Multiple Data，一条指令处理多个数据。实践卷后续会用它优化热循环。

I/O/EOF I/O 是 Input/Output，输入输出；**EOF** 是 End Of File，表示输入结束。实践卷会把读取、短写、完成事件和业务提交分开处理。

CLI/CI CLI 是命令行接口，便于脚本化运行实验；CI 是持续集成，用自动构建和测试保护项目边界。

JSON/CSV JSON 适合保存结构化配置和 **manifest**；CSV 适合保存表格化 **benchmark**、**trace** 摘要和对比结果。

manifest 机器可读清单，记录输入版本、输出位置、校验和、参数和提交事实。

checkpoint 检查点，保存可恢复状态，让失败后可以从已提交边界继续。

attempt 一次物理尝试。同一个逻辑任务可能因为失败重试而产生多个 **attempt**。

epoch 版本或轮次编号，用来区分不同时刻的状态和提交边界。

MessageBus 消息总线。本卷里指组件之间传递任务、完成事件和控制消息的工程边界。

batch/split **split** 是输入切分后的逻辑片段；**batch** 是热路径里成批处理的一组记录。**split** 偏调度身份，**batch** 偏执行效率。

idempotent 幂等。重复执行同一逻辑请求不会产生额外业务结果，是重试、**manifest** 和恢复的核心要求。

benchmark/trace **benchmark** 是可重复的性能测量；**trace** 是按时间记录的事件流。实践卷里二者都要带输入版本、构建选项和结果校验。

target/span/event/generation **target** 是 CMake 构建目标或业务目标；**span** 是一段连续数据的

非拥有视图；event 是状态变化或完成事实；generation 是同一逻辑输入的新版本编号。

offset/line number offset 是输入中的字节偏移，line number 是行号。诊断同时保存二者，是为了既能机器重放，也能让人快速定位原始文本。

工程对象名 **ManifestEntry** 表示清单中的一条输入或输出记录；**InputSplit** 表示输入被切出来的逻辑片段；**HotBatch** 表示热路径中被成批处理的数据；**QueueContract** 表示队列必须满足的容量、顺序、关闭和失败语义；**RunReport** 表示一次运行的结构化结果。

诊断字段名 **input_name** 是输入来源名；**input_generation** 是输入版本；**byte_offset** 是字节位置；**line_number** 是行号；**error_kind** 是错误分类；**records_total** 是处理记录总数。这些字段的目标是让失败能被复现，而不是让日志看起来复杂。

UTF-8 Unicode 的常见变长字节编码。实践卷保存 offset 时默认按字节定位；如果 UI 要按“第几个字符”显示，就必须额外处理 UTF-8 解码，不能把字节下标直接当字符下标。

全书结构

本卷按 Compute Systems Engine 的工程演化组织。第一部分建立输入合同、reference、诊断和测试框架。第二部分加入数据布局、热路径和 benchmark harness。第三部分加入并行内核、线程池、同步和原子协议。第四部分加入任务图、运行时、异步 I/O、可靠写和 checkpoint。第五部分加入 MessageBus、attempt、epoch、分片、复制和端到端故障注入。

每个阶段都必须产出四类东西。第一，稳定接口，例如 `InputSplit`、`RecordId`、`HotBatch`、`QueueContract`、`ManifestEntry` 或 `RunReport`。第二，可运行实现，至少包含 reference 和一个带机制的版本。第三，测试，包括正常输入、边界输入和能击穿朴素方案的失败输入。第四，报告，记录正确性、性能、资源、故障和未验证边界。

本卷与第二册原理卷并行阅读。原理卷先解释机制为什么存在，本卷再把机制落成代码。若读者只读本卷而跳过原理卷，容易把接口当成固定答案；若只读原理卷而不做本卷，容易把工程边界停留在口头推导。两卷合起来，才形成从零到精通的完整训练。

目录

常见缩写和术语先导	iii
第一部分 从 reference 到可测试工程	
第 1 章 输入合同、Reference 与第一组测试	2
第二部分 Reference Pipeline 实践	
第 2 章 实践卷：Reference Pipeline 工程化	5

第零部分

从 reference 到可测试工程

第 1 章

输入合同、Reference 与第一组测试

1.1 本章目标

本章实现 Compute Systems Engine 的第一个可运行切片：输入合同、顺序 reference、诊断结果、稳定输出和测试。它对应原理卷第一章的内容。此时不做并行、不做 SIMD、不做 manifest，也不追求吞吐；目标只有一个：让项目拥有一把可信尺子。后续所有优化都必须先通过这把尺子。

先把本章几个词落到代码对象上。合同不是抽象口号，而是函数边界的承诺：输入记录长什么样，缺字段算什么错误，输出 key 是否排序，计数溢出是否允许。reference 是最朴素但可信的实现，用来定义正确结果；后面 SIMD、线程池或异步 I/O 版本都必须和它对齐。diagnostic 是结构化错误事实，不是随手打印的日志；它要能指出哪条记录、哪个字段、为什么失败。checksum 是输出摘要，用来证明同一输入、同一合同下结果稳定。

第一版工程应至少包含四个模块：

模块	职责	不负责
<code>input</code>	读取固定测试输入，分配 <code>RecordId</code>	并行切分和异步 I/O
<code>reference</code>	顺序解析和计数	优化、缓存布局和线程
<code>report</code>	输出 <code>accepted</code> 、 <code>rejected</code> 、 <code>counts</code> 和 <code>checksum</code>	性能归因
<code>tests</code>	固定正常输入、坏行和稳定输出	压力 benchmark

1.2 第一版接口

第一版接口要小，但不能含糊。建议先定义四个概念：`RecordId`、`ParseDiagnostic`、`ReferenceResult` 和 `InputCase`。完整字段可以随项目扩展，但第一版必须能回答三件事：哪条记录被接受，哪条记录被拒绝，最终计数是否稳定。

Listing 1.1 第一版核心结构

```
1 struct RecordId {
2     std::string input_name;
3     std::uint64_t input_generation = 0;
4     std::uint64_t byte_offset = 0;
5     std::uint64_t line_number = 0;
6 };
7
8 struct ParseDiagnostic {
9     RecordId record;
10    std::string error_kind;
11 };
12
13 struct ReferenceResult {
14     std::uint64_t accepted = 0;
15     std::vector<ParseDiagnostic> diagnostics;
16     std::map<std::string, std::uint64_t> counts;
17 };
```


1.3 测试先于优化

第一组测试不追求覆盖所有未来功能，只固定最核心语义。正常五行输入应得到 `view=2`；缺字段输入应进入 diagnostics；字段过多、空事件名和 value 非整数都应有明确错误类型；输出 key 顺序必须稳定。若这些测试不稳定，后续任何性能实验都应暂停。

习题与作业

本章实现任务：建立 CMake target、顺序 reference 和测试。测试至少包含 normal five lines、missing field、extra field、empty event name、bad value 五类输入。每个测试同时检查 accepted、diagnostics、counts 和稳定 checksum。

1.4 报告格式

第一版报告可以是简单 key-value 文本，不需要复杂 UI。最低字段包括输入名、输入版本、总行数、accepted、rejected、输出 checksum 和错误类型计数。报告不是给人截图看的，而是后续脚本和读者复查用的证据。

Listing 1.2 第一版报告示例

```
input_name=normal_5_lines
input_generation=1
records_total=5
accepted=5
rejected=0
checksum=...
count.click=1
count.login=1
count.logout=1
count.view=2
```

本章结束时，项目还很小，但它已经具备工程地基：语义明确、测试可跑、报告可查。下一章才能开始讨论 split 和 RecordId 在并行输入中的作用。

第零部分

Reference Pipeline 实践

第 2 章

实践卷：Reference Pipeline 工程化

上一章定义了输入合同、reference 和报告字段。本章把它落成可构建工程，目录是：

```
1 books/compute-systems-engine-code/labs/reference_pipeline
```

代码使用 C++20，单元测试使用 GTest/GMock，性能入口使用 Google Benchmark。这里的重点不是工具名，而是把输入身份、解析诊断、稳定报告和 manifest 发布格式做成可复查工程。

2.1 一、工程入口

核心文件如下：

```
1 include/cse/reference_pipeline.hpp
2 src/reference_pipeline.cpp
3 src/main.cpp
4 tests/reference_pipeline_tests.cpp
5 bench/reference_pipeline_bench.cpp
```

构建、测试和 benchmark smoke 命令如下：

Listing 2.1 代码实践卷 reference pipeline 构建

```
1 cmake -S books/compute-systems-engine-code -B books/compute-systems-engine-code ↵
   ↵ /build/reference-debug -DCMAKE_BUILD_TYPE=Debug
2 cmake --build books/compute-systems-engine-code/build/reference-debug
3 ctest --test-dir books/compute-systems-engine-code/build/reference-debug -- ↵
   ↵ output-on-failure
4 books/compute-systems-engine-code/build/reference-debug/labs/reference_pipeline ↵
   ↵ /cse_reference_bench --benchmark_min_time=0.01s
```

2.2 二、阶段 0：输入身份先独立成类型

RecordId 描述一条记录从哪里来：

```
1 struct RecordId {
2     std::string input_name;
3     std::uint64_t input_generation = 0;
4     std::uint64_t byte_offset = 0;
5     std::uint64_t line_number = 0;
6 };
```

input_name 是输入名，**input_generation** 是同一输入的版本号，**byte_offset** 是该行在原始文本里的字节偏移，**line_number** 是行号。generation 不是时间戳装饰，它用于区分“同名输入的第几代”。如果后续做 retry、checkpoint 或恢复，旧 generation 的结果不能覆盖新 generation。

InputCase 把输入身份和文本放在一起。实践时不要直接传裸字符串，因为裸字符串没有来源，坏行诊断也无法说明是哪份输入、哪一代、哪个位置出错。

2.3 三、阶段 1：解析合同从一行开始

reference pipeline 的输入格式是每行 **event,value**。合法行有两个字段：事件名非空，value 是整数。坏行进入 **ParseDiagnostic**，不能混入 accepted 计数。

手算样例：

```
1 view,1
2 click,1
3 view,2
```

这三行的 **records_total** 是 3, **accepted** 是 3, **counts["view"]** 是 2, **counts["click"]** 是 1。

坏行样例：

```
1 missing
2 click,1,extra
3 ,2
4 login,abc
```

它们分别对应 **missing_field**、**extra_field**、**empty_event** 和 **bad_value**。测试 **BadLinesReturnStructuredDiagnostics** 不只检查错误个数，还检查 line number、error kind 和 byte offset。这样后续换 parser 时，不能把诊断精度降级。

2.4 四、阶段 2：reference 先可信，再考虑快

run_reference 是可信但不追求快的实现。它逐行扫描输入，只有合法记录才更新 **counts** 和 checksum。checksum 是结果摘要，用来证明同一输入反复运行稳定，也用于后续优化版对照。

Listing 2.2 reference result 的语义字段

```
1 struct ReferenceResult {
2     std::string input_name;
3     std::uint64_t input_generation = 0;
4     std::uint64_t records_total = 0;
5     std::uint64_t accepted = 0;
6     std::vector<ParseDiagnostic> diagnostics;
7     std::map<std::string, std::uint64_t> counts;
8     std::uint64_t checksum = 0;
9 };
```

这里使用 **std::map** 是为了让输出 key 顺序稳定。稳定顺序不是小事：报告、checksum、manifest 和回归测试都依赖它。等后续进入热路径优化，可以用哈希表或分片结构提高吞吐，但发布报告前仍要回到稳定顺序。

2.5 五、阶段 3：报告和 manifest 是发布合同

format_report 面向人读，必须包含输入名、记录数、accepted、checksum、每个事件计数和错误统计。**ManifestRow** 面向机器读，字段固定为 input name、generation、records total、accepted、rejected 和 checksum。

manifest 是发布清单。它回答“这次处理发布了哪份输入、哪一代、多少记录、多少接受、多少拒绝、摘要是什么”。后续可靠写、恢复和分布式提交都会依赖它；因此 manifest 格式不能随手改。测试 **ReportAndManifestAreDeterministic** 固定了 report 子串和 manifest 前缀。

2.6 六、阶段 4: 测试覆盖正常、坏行、空输入和诊断质量

当前测试分五类:

1. **NormalInputCountsAndChecksumAreStable**: 正常五行输入, 检查 counts 和 checksum 稳定;
2. **BadLinesReturnStructuredDiagnostics**: 四类坏行, 检查错误类型和位置;
3. **ReportAndManifestAreDeterministic**: 检查报告和 manifest 输出稳定;
4. **EmptyInputHasZeroRecordsButStableChecksum**: 空输入没有记录但 checksum 稳定;
5. **DiagnosticDetailNamesTheContract**: 诊断 detail 必须说清合同, 例如 “exactly two fields”。

测试不是附属品。reference pipeline 后续每次优化, 都必须先跑这些测试, 再用优化结果对照 reference。若优化后报告字段变少、错误位置变粗或 checksum 不稳定, 即使吞吐更高, 也不能进入书中的 “正确实现”。

2.7 七、阶段 5: benchmark 只测 reference 主路径

bench/reference_pipeline_bench.cpp 只测 **run_reference** 在不同记录数下的成本。Debug benchmark 只能说明 target 可运行; 正式性能报告必须使用 Release 构建, 并记录 CPU、编译器、输入规模、负载、是否开启 CPU scaling、重复次数和异常样本处理。

不要把文件读取、随机输入生成、报告打印和 parser 统计混在同一个 benchmark。一次 benchmark 只回答一个问题: 在已给定输入文本的情况下, **run_reference** 的成本怎样随记录数增长。

2.8 八、下一阶段: input split

下一步扩展是 input split。split 是把大输入切成多个片段并行处理的单位。它不能破坏 **RecordId**: 每条记录仍要保留 input name、generation、byte offset 和 line number。

正确实现必须处理 “split 边界落在行中间” 的情况。不能让前一个 split 和后一个 split 同时处理同一行, 也不能让两边都跳过这行。测试应构造一个长输入, 把切点放在 **event, value** 中间, 然后证明总记录数、accepted、诊断行号和 checksum 与整文件 reference 一致。

2.9 九、项目阶段清单

阶段	代码切片	验收证据
0	RecordId/InputCase	每条记录有输入名、generation、offset、line
1	line parser	四类坏行进入结构化 diagnostics
2	run_reference	accepted、counts、checksum 稳定
3	report/manifest	人读报告和机器清单格式固定
4	GTest/GMock tests	正常、坏行、空输入、诊断质量全覆盖
5	benchmark	只测 run_reference , 不混入 I/O
6	input split 扩展	split 落在行中间不丢行、不重行、不错行号

下一步扩展: 把 input split 加入工程。要求每个 split 保留 **RecordId** 的 input name、generation、byte offset 和 line number; 并写一个测试证明 split 边界落在行中间时不会丢行、重行或错报 line number。

术语表

Compute Systems Engine 本卷贯穿项目，用于承接第二册原理卷中的硬件、并发、运行时、I/O 和分布式计算机制。

reference 参考实现，用清楚、确定、可复查的方式定义正确结果。

oracle 判定器，用于比较 reference 和待测实现的输出是否满足同一语义。

contract 合同，代码边界的明确约定：输入、输出、错误、前置条件和不承诺行为。

RecordId 记录身份，用 input name、generation、byte offset 和 line number 定位一条输入。

diagnostic 结构化诊断事实，说明哪条记录为什么被拒绝或哪条边界失败。

checksum 校验和，把输出内容压成摘要，用于稳定性检查、截断检查和重放对比。

CPU Central Processing Unit，中央处理器。本卷先把实现、测试和 benchmark 落在本地 CPU 环境。

UI User Interface，用户界面。实践卷里主要指用户看到的诊断、报告和工具输出。

C++20 C++ 标准版本名。本卷默认使用的语言和标准库能力边界。

CMake C++ 工程常用构建系统，用于组织 target、测试、benchmark 和工具程序。

std:: C++ 标准库命名空间前缀。**std::vector** 表示拥有元素的动态数组，**std::string** 表示字节字符串，**std::map** 表示按键排序的关联容器，**std::uint64_t** 表示固定 64 位无符号整数。

InputCase 第一章示例中的一条输入用例，通常包含原始文本、来源和可重放身份。

ReferenceResult 参考实现产生的结构化输出，用来喂给 oracle 做正确性判定。

ParseDiagnostic 解析阶段产生的结构化错误事实，包含错误类别、位置和可读说明。

SIMD Single Instruction, Multiple Data，一条指令处理多个数据。

I/O Input/Output，输入输出。异步 I/O 中 submit、completion 和 commit 不是同一件事。

EOF End Of File，输入结束。读取代码要区分 EOF、短读和错误。

CLI Command-Line Interface，命令行接口，用参数、标准输入输出和退出码表达工具行为。

CI Continuous Integration，持续集成，用自动构建、测试和检查保护每次提交。

JSON JavaScript Object Notation，结构化文本格式，适合保存配置、manifest 和报告。

CSV Comma-Separated Values，逗号分隔文本表格，适合保存实验结果和统计数据。

manifest 机器可读清单，记录输入版本、输出位置、校验和、参数和提交事实。

checkpoint 检查点，保存可恢复状态，让失败后可以从已提交边界继续。

attempt 一次物理尝试。同一个逻辑任务可能因为失败重试而产生多个 attempt。

epoch 版本或轮次编号，用来区分不同时刻的状态和提交边界。

MessageBus 消息总线，用于组件之间传递任务、完成事件和控制消息。

split 输入切分后的逻辑片段，通常决定调度、重试和 attempt 身份。

batch 热路径里成批处理的一组记录，通常服务缓存局部性、SIMD 或线程分摊。

idempotent 幂等，重复执行同一逻辑请求不会产生额外业务结果。

offset 输入中的字节偏移，用于机器重放和精确定位。

line number 输入文本中的行号，用于人阅读诊断和快速定位错误。

索引

Compute Systems Engine, ii